An exact null for the Rasch change index

Why the RCI critical value is not 1.96, and how to compute it without simulating

Magnus Johansson

2026-09-12

Table of contents

Purpose

`RMpersonChange()` includes an option for `critical = "simulate"` rather than the conventional 1.96, because the Rasch change index

$$RCI = \frac{\hat{\theta}_2 - \hat{\theta}_1}{\sqrt{SE(\hat{\theta}_1)^2 + SE(\hat{\theta}_2)^2}}$$

is not standard normal under the null when the scale is short (or, as we will see, when the scale is < ~80 items). This note shows that the null does not need simulating at all: for complete data it can be enumerated exactly. It also records what the exact critical value does across test length, sample skew and targeting, and one consequence for the RCI as a statistic.

Why the null is not N(0, 1)

Two mechanisms, both weakening as test information grows.

Self-normalisation. $SE(\hat{\theta}_1)$ and $SE(\hat{\theta}_2)$ are deterministic functions of the realised scores, so the denominator is not independent of the numerator. A respondent who moves to an extreme score produces a large numerator and a large denominator together, and the ratio is damped. Both tails of the null are pulled in, so the critical value sits **below** 1.96. In a t-test the denominator is independent of the numerator; here it is not, which is why the t reference does not help either.

Discreteness. With complete data the sum score is sufficient, so $\hat{\theta}$ takes only as many values as there are scores and the null is discrete rather than continuous.

Exact enumeration

Sufficiency is what makes this tractable. $\hat{\theta}(r)$ and $SE(r)$ are deterministic lookups, the ones `RMscoreSE()` builds, and $P(r \mid \theta)$ follows from the Lord-Wingersky recursion. Under the null the two occasions are independent given θ, so the joint is $P(r_1 \mid \theta)P(r_2 \mid \theta)$ and the whole null is a discrete distribution over $(k+1)^2$ score pairs. For six items with four response categories that is 361 cells.

```
# One threshold constructor for the whole note: k items, ncat response
# categories, item locations spread over [-1.2, 1.2].
mk <- function(k, ncat) {
  lapply(seq(-1.2, 1.2, length.out = k),
         function(b) b + seq(-0.9, 0.9, length.out = ncat - 1L))
}

# Lord-Wingersky: exact sum-score distribution given theta under a PCM
score_dist <- function(thr_list, theta) {
```

```
  d <- 1
  for (thr in thr_list) {
    p <- easyRasch2:::.pcm_cat_probs(theta, thr)
    nd <- numeric(length(d) + length(p) - 1L)
    for (a in seq_along(d)) {
      nd[a + seq_along(p) - 1L] <- nd[a + seq_along(p) - 1L] + d[a] * p
    }
    d <- nd
  }
  d                                    # index is score + 1
}

# Exact null distribution of the RCI, pooled over a set of person locations
exact_crit <- function(thr_list, thetas, alpha = 0.05) {
  steps <- vapply(thr_list, length, integer(1L))
  lut <- vapply(0:sum(steps), function(r) {
    easyRasch2:::.theta_wle(
      easyRasch2:::.score_pattern(r, steps), thr_list, c(-10, 10)
    )
  }, numeric(2L))
  th <- lut[1L, ]
  se <- lut[2L, ]
  rci <- outer(seq_along(th), seq_along(th),
               function(i, j) (th[j] - th[i]) / sqrt(se[i]^2 + se[j]^2))
  w <- matrix(0, length(th), length(th))
  for (t in thetas) {
    d <- score_dist(thr_list, t)
    w <- w + outer(d, d)
  }
  w <- w / sum(w)
  o <- order(as.numeric(rci))
  v <- as.numeric(rci)[o]
  cw <- cumsum(as.numeric(w)[o])
  c(lower = v[which(cw >= alpha / 2)[1L]],
    upper = v[which(cw >= 1 - alpha / 2)[1L]])
}
```

Validation against simulation

Run by `dev/` scratch scripts at `sim_iter` 2000 to 4000, n = 200, alpha = .05 two-sided. The simulated and exact values agree to within Monte Carlo error in every configuration tried, including dichotomous items, items with unequal category counts, a right-skewed person distribution and a badly off-target one.

configuration	simulated	exact	difference
6 items, 4 categories, symmetric theta	−1.695 / +1.695	±1.695	0.000
6 items, 4 categories, skewed theta	−1.725 / +1.695	±1.695	0.030
6 items, 4 categories, skewed and off target	−1.534 / +1.534	±1.522	0.012
20 items, 2 categories, symmetric	−1.798 / +1.798	±1.798	0.000
20 items, 2 categories, skewed	−1.798 / +1.798	±1.798	0.000
9 items, mixed 2/3/4 categories, symmetric	−1.721 / +1.721	±1.721	0.000
9 items, mixed 2/3/4 categories, skewed	−1.721 / +1.721	±1.721	0.000

configuration	simulated	exact	difference
12 items, 4 categories, symmetric	−1.864 / +1.864	±1.862	0.002

The exact values are perfectly symmetric. That is the expected result: under the null the two occasions are exchangeable, so $\hat{\theta}_2 - \hat{\theta}_1$ is symmetric about zero however skewed $\hat{\theta}$ itself is. The residual asymmetry in the simulated column is Monte Carlo noise. Asymmetry becomes real only when exchangeability fails, which in practice means the two occasions differing in which items were answered.

The critical value tracks test information

```
set.seed(1)
theta <- rnorm(400, 0, 1.4)
tab <- t(vapply(c(4, 6, 10, 20, 40), function(k) {
  c(items = k, exact_crit(mk(k, 4), theta)[["upper"]])
}, numeric(2L)))
kable(tab, digits = 3, col.names = c("Items", "Critical value"))
```

Items	Critical value
4	1.644
6	1.725
10	1.815
20	1.868
40	1.915

Monotone in test length, converging on 1.96, with the number of response categories held at four throughout. Referring the RCI to a normal null is therefore conservative, and increasingly so the shorter the scale. At six items and four categories, using 1.96 instead of 1.695 is a materially stricter test.

Response categories have to be reported alongside item count, because a six-item scale with three categories and one with seven are not comparable instruments. They do not act on the two quantities in the same way:

```
cats <- t(vapply(c(2, 3, 4, 5, 7), function(nc) {
  thr <- mk(6, nc)
  names(thr) <- paste0("I", seq_along(thr))
  d <- as.data.frame(easyRasch2:::sim_partial_score(thr, theta))
  colnames(d) <- names(thr)
  rel <- RMreliabilityCurve(d, item_params = thr, output = "dataframe")
  c(categories = nc,
    critical = exact_crit(thr, theta)[["upper"]],
    marginal = attr(rel, "marginal_ratio"))
}, numeric(3L)))
kable(cats, digits = 3,
      col.names = c("Categories", "Critical value", "Marginal reliability"))
```

Categories	Critical value	Marginal reliability
2	1.546	0.618
3	1.724	0.764
4	1.725	0.829
5	1.727	0.872
7	1.779	0.906

Reliability climbs steeply with categories, from about .60 at two to about .91 at seven. The critical value does not follow it. Almost all of its movement is the step from dichotomous to polytomous, after which it is close to flat. So categories buy precision without much changing how far the null sits from 1.96, while items move both. There is a section below with further discussion on reliability and conditional reliability.

Skew and targeting

Sample skew on its own barely moves the pooled critical value: the skewed and symmetric rows above agree to the third decimal in five of seven configurations. Targeting is what matters. Pushing the same skewed distribution 2.2 logits off target, so that 39.5% of respondents sit at an extreme score at one occasion or the other, drops the critical value from 1.695 to 1.522. The mechanism is the self-normalisation above: a floor-bound sample is located primarily where the statistic is most heavily damped.

This also means the pooled critical value is a property of the sample as well as the items, which is worth stating wherever it is reported.

A consequence for the RCI as a statistic

Because the denominator is a function of the same scores as the numerator, the RCI does not order respondents by how much they changed. A concrete pair from the six-item scale above:

	score change	change in logits	RCI
A	$0 \longrightarrow 18$	7.24	3.53
B	$3 \longrightarrow 15$	3.11	3.60

Respondent A moved the full width of the scale, 4.14 logits further than B, and scores lower on the RCI. Both are far past any critical value so nothing turns on it here, but it shows that the RCI is a standardised distance from the null rather than a measure of change, and should not be read as the latter.

The constructive version: since the critical value has to be computed from the item parameters anyway, the normalisation buys nothing that a location-specific threshold on the raw change would not. The same enumeration gives the null of $\hat{\theta}_2 - \hat{\theta}_1$ directly, so a conditional critical **change**, in logits, is available at no extra cost and is monotone in what happened. That is the same object as a conditional minimal detectable change, and connects to the score-level MDC table parked as a possible sibling function.

The no-change band in the figure

The scatter plot of occasion 1 against occasion 2 draws a band covering the changes that would not be flagged. That band was built from a smooth approximation, half-width $c \cdot \sqrt{2/I(\theta)}$, which places **both** occasions at the baseline location. The test does not do that. It uses $SE(r_1)$ and $SE(r_2)$ at two different scores.

On 300 respondents the two disagreed for 33 of them, 11 percent: 27 sat outside the drawn band without being flagged, and 6 sat inside it while flagged. The figure contradicted the table it accompanied.

The mechanism shows in the worst case. A respondent moving from $\theta_1 = 1.23$ to $\theta_2 = 3.62$ had a band built on $SE_{diff} = 0.79$, because information is high at 1.23, while the test used 1.56, because the follow-up score sits at the ceiling. The band was about half the width it should have been, in precisely the case the figure exists to display: a large change landing near a boundary.

The band is now built from the same lookup the test uses. For each attainable baseline score, the follow-up locations that would not be flagged are read off directly, so a point lies outside the band if and only if that respondent was flagged. Verified at zero disagreements out of 300 and pinned by a regression test. Because attainable scores are discrete the band is a step function, which also makes the discreteness visible rather than hiding it behind a smooth curve.

```
set.seed(1)
thr_demo <- mk(6, 4)
names(thr_demo) <- paste0("I", seq_along(thr_demo))

theta_demo <- rnorm(300, 0, 1.6)
t1 <- as.data.frame(easyRasch2:::sim_partial_score(thr_demo, theta_demo))
t2 <- as.data.frame(easyRasch2:::sim_partial_score(thr_demo, theta_demo + 0.7))
colnames(t1) <- colnames(t2) <- names(thr_demo)

# Item parameters are estimated from the data here, as a reader would, so that
# this figure and the reliability output below share one calibration.
RMpersonChange(t1, t2, critical = "exact", output = "ggplot")
```

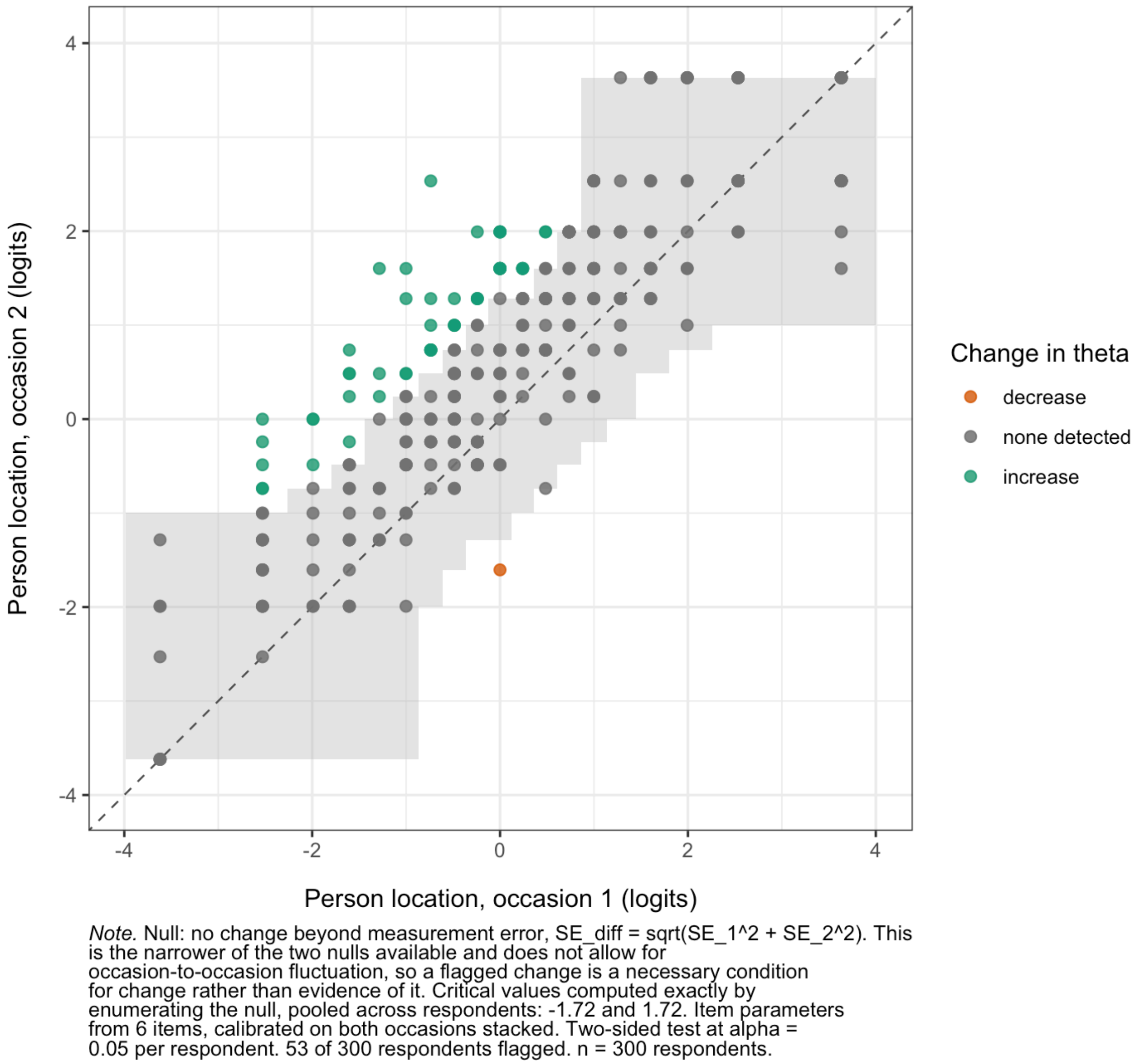


Figure 1: Default `RMpersonChange()` figure. Three hundred respondents on a scale of six items with four response categories, generated with a true shift of 0.7 logits between occasions. The grey band covers the follow-up locations that would not be flagged from each baseline score, so a point lies outside it if and only if that respondent was flagged. Its steps are the attainable scores, and it is widest away from the centre.

The band table below gives the same object numerically.

```
thr6 <- mk(6, 4)
crit <- exact_crit(thr6, theta)

band_rows <- function(thr_list, lo, hi) {
  steps <- vapply(thr_list, length, integer(1L))
  lut <- vapply(0:sum(steps), function(r) {
    easyRasch2:::.theta_wle(
      easyRasch2:::.score_pattern(r, steps), thr_list, c(-10, 10)
    )
  }, numeric(2L))
  th <- lut[1L, ]
  se <- lut[2L, ]
  do.call(rbind, lapply(seq_along(th), function(i) {
    rci <- (th - th[i]) / sqrt(se[i]^2 + se^2)
```

```
    keep <- rci <= hi & rci >= lo
    data.frame(baseline = th[i], lower = min(th[keep]), upper = max(th[keep]))
  }))
}

b <- band_rows(thr6, crit[["lower"]], crit[["upper"]])
b$width <- b$upper - b$lower
kable(b, digits = 2, row.names = FALSE,
      col.names = c("Baseline theta", "Lower", "Upper", "Width"))
```

Baseline theta	Lower	Upper	Width
−3.73	−3.73	−1.27	2.46
−2.58	−3.73	−0.98	2.74
−2.01	−3.73	−0.72	3.01
−1.60	−3.73	−0.47	3.25
−1.27	−3.73	0.00	3.73
−0.98	−2.58	0.23	2.82
−0.72	−2.01	0.47	2.48
−0.47	−1.60	0.72	2.32
−0.23	−1.27	0.98	2.26
0.00	−1.27	1.27	2.55
0.23	−0.98	1.60	2.59
0.47	−0.72	1.60	2.32
0.72	−0.47	2.01	2.48
0.98	−0.23	2.58	2.82
1.27	0.00	3.73	3.73
1.60	0.23	3.73	3.49
2.01	0.72	3.73	3.01
2.58	0.98	3.73	2.74
3.73	1.27	3.73	2.46

The width is not monotone in distance from the centre, which is worth knowing before reading a figure. It peaks off centre, where the standard error is already large but the scale has not run out of room, and narrows again at the boundary itself because there are no further attainable scores to move to. For this scale, six items with four categories, the widest band is 3.62 logits at a baseline near ± 1.2, against 2.47 at the centre and 2.39 at the extremes.

Two cases remain indicative rather than exact, and are documented as such: with mixed missingness the band is drawn for the most common pair of answered-item sets, and under `conditional_crit = TRUE` it uses the median critical values.

What the change figure is resting on

A change verdict is only as good as the precision behind it, and the reader of the figure above has no way to judge that from the figure alone. The same scale, six items with four response categories, summarised the two ways the package offers.

```
RMreliability(t1, draws = 200, rmu_iter = 10, seed = 1, boot = TRUE)
```

Metric	Estimate	Lower (95% HDCI)	Upper (95% HDCI)	Notes
Cronbach's alpha	0.868	0.850	0.889	200 bootstrap resamples
PSI	0.796	0.772	0.820	200 bootstrap resamples
Marginal (curve mean)	0.856	0.842	0.868	200 bootstrap resamples

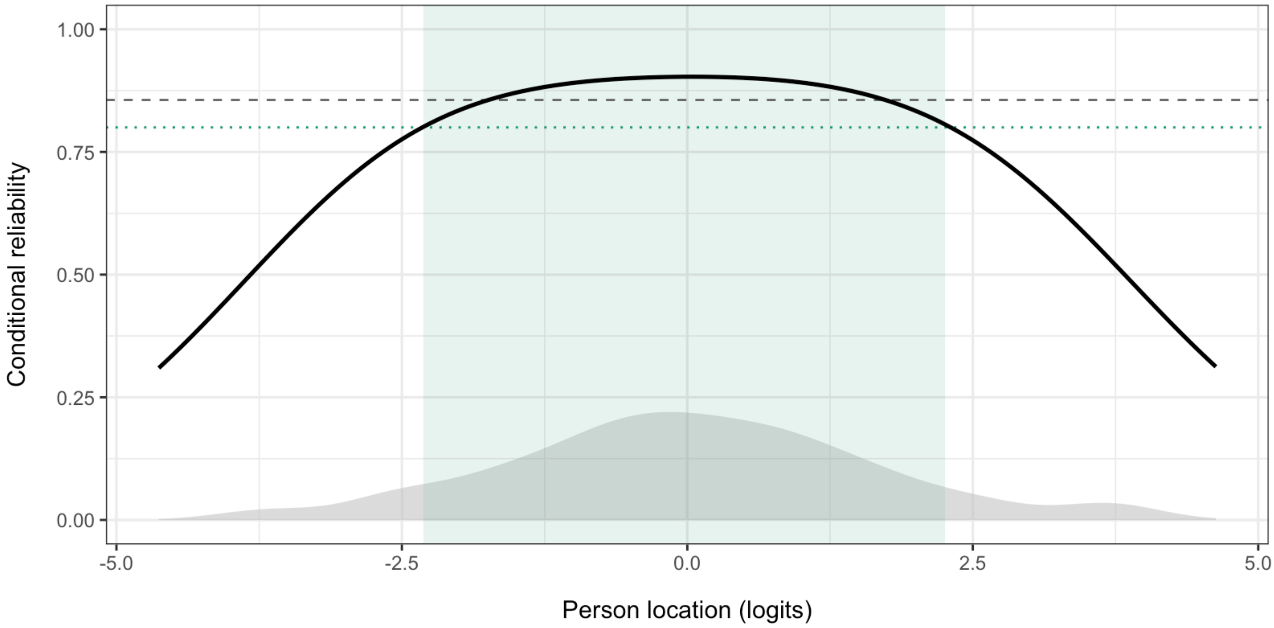
Metric	Estimate	Lower (95% HDCI)	Upper (95% HDCI)	Notes
RMU (WLE)	0.861	0.839	0.882	200 PVs, 10 RMU iterations

Marginal reliability near .85 is respectable for six items with four response categories, and the gap down to PSI is the ordinary one between a model-based coefficient integrated over the fitted latent density and a sample-based one computed from the observed spread of the estimates.

The "Marginal (curve mean)" row and the dashed line in the curve below are the same number, the latent-density-weighted mean of the conditional reliability curve. `RMreliabilityCurve()` returns it as the `marginal_ratio` attribute, and the two agree to machine precision whenever both are given the same data and the same calibration.

A single coefficient hides where that precision sits, which is the same point the stepped band makes in the change figure.

```
RMreliabilityCurve(t1, statistic = "reliability", benchmark = 0.8)
```



Note. Conditional reliability, sigma^2 / (sigma^2 + SEM(theta)^2) with sigma = 1.54, from test information summed over 6 items using CML thresholds. The curve is a property of the items, not of the sample. Respondent locations by WLE. Shaded distribution shows where respondents fall. Dashed line is the marginal reliability (0.856), the flat summary a single coefficient reports. Dotted line and shaded band mark reliability >= 0.80, reached by 83% of respondents. Information-based standard errors are asymptotic and only approximate for a scale this short. n = 300 respondents.

Figure 2: Conditional reliability across the scale for the same six items with four response categories, with the region reaching .8 shaded and the respondent distribution behind it.

Read Figure 1 and 2 together. Where this curve is high the band in the change figure is narrow and a modest change is detectable; where it falls away the band widens and only a large change clears it. The band narrowing again at the very extremes is not precision returning, it is the scale running out of scores.

Scope and limits

- **Complete data only.** Missing data breaks sufficiency of a single sum score. Each distinct missingness pattern has its own score distribution and its own lookup, so the enumeration still works but per pattern.
- **`null = "retest"` needs quadrature.** The occasion deviations are continuous, so the joint becomes a double integral over u_1, u_2 rather than a pure sum. Still deterministic and cheap at, say, 21 nodes a side.
- **EAP is fine.** On complete data an EAP estimate is also a function of the sum score.
- Validated here for WLE with CML thresholds. Not yet checked against MML item parameters.

Implemented

Shipped as `critical = "exact"`, now the default in `RMpersonChange()`, in `R/person_change_exact.R`. Measured against a 4000-iteration simulation on the same data: critical values agree to within Monte Carlo error, p-values correlate at 0.9998, classifications are identical, and the exact path takes 0.08 s against 13.7 s. Incomplete data is enumerated within each missingness pattern, `null = "retest"` integrates the occasion deviations out by quadrature (they are independent, so the joint factorises and the retest null costs no more than the measurement null), and EAP is supported by holding the prior at the one the observed respondents were scored under.

Having an exact reference immediately paid for itself twice. It exposed a bug in the simulation path, which had been re-estimating the EAP prior from each null dataset rather than holding it at the observed one. That is now fixed, and the two paths agree to four decimals under EAP. It also supplied the lookup needed to fix the figure, described in the next section.

It also removes the one real objection to `conditional_crit = TRUE`: a per-respondent critical value from simulation rests on `sim_iter` draws, roughly 25 of which land in each tail at alpha = .05, whereas $P(r \mid \theta_i)$ is available per respondent for free.

It would also sharpen the planned simulation study, whose first aim is the realised Type I error of the simulated critical value. Much of the noise in that comparison disappears if the reference is exact.

Reproduction

Scratch scripts used for the simulated column are not part of the package. The exact computation is self-contained in the `exact-fns` chunk above and needs only `easyRasch2:::.pcm_cat_probs()`, `:::.theta_wle()` and `:::.score_pattern()`.